



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Let's review data types and characteristics of C#.



Learning Outcome:

Talk about principles, concepts, characteristics of general-purpose computer programming language in spoken and written situations.

Can Do Statements:

- I can list different kinds of data in C#.
- I can talk about characteristics of different kinds of data in C#.
- I can recall some value types in C#.

Ask your partner:

- What do you know about data?
- What is a dynamic type?





Let's Explore!

Alan does not understand this topic.



Instructions: Read the text below.

- Alan:** Carla, I feel like there must be something wrong with me; I still can't get my head around this C# stuff!
- Carla:** What exactly is it that you don't get?
- Alan:** Well, I obviously understand that it's a versatile general-purpose computer programming language that closely follows the C and C++ languages, but when it comes to the data types I'm totally lost!
- Carla:** Oh well, the important ones are really quite simple. Let's start with value types; they allow us to store data directly, such as numbers, alphabets, and floating point numbers, and they can be directly assigned a value.
- Alan:** Ok, I can see how they would be useful.
- Carla:** Then you have the reference type. The reference type does not store data directly but contains references to all of the variables. In other words, it refers to the memory location.
- Alan:** So it's like an indexed file system that allows you to look up for the data you need.
- Carla:** Yes, something like that. Then there's the pointer type. Pointer type is very easy for anyone who can use C or C++ because they have the same capabilities – they store the address of another type.
- Alan:** All of those seem easy enough. What about object, dynamic, and string types?
- Carla:** Object type is probably the most important of those three. It is the ultimate base class for all types in C# common systems. It can be assigned the values of any other data types. However, before assigning values, it needs converting. This conversion is called "boxing," or "unboxing" in the other direction.
- Alan:** That's a seriously useful data type!
- Carla:** It certainly is. The dynamic type is also useful as you can store any data type inside it. Finally string type; this type allows you to assign any string values to a variable, making larger chunks of data more accessible.

**Let's Practice!**

Alan is planning his work schedule.

Instructions: Proofread the text. Correct the mistakes.

Now I am work with different kinds of data, and I'm currently organize my schedule. I go to search for more information about reference type to work with this data. Then, I am go looking for the way to use the pointer type to store addresses from another type. Now that I think it better, I wheel also read more about value types in C# to learn the way this data works.

**Let's Practice!**

Which type could be the best for these cases?

Instructions: Read the question. Select the correct answer.

1. What does the C# allow you to do?
 - a. Follow the Java language.
 - b. Follow the Windows language.
 - c. Follow the C and C++ languages.
 - d. Follow the ASP.Net language.
2. Which value type in C# allows you to store data such as numbers, alphabets, and floating-point numbers in a direct way?
 - a. Reference type.
 - b. Value types.
 - c. Pointer type.
 - d. Object type.
3. Which value type in C# allows you to store the address of another type?
 - a. Reference type.
 - b. Object type.
 - c. Value types.
 - d. Pointer type.
4. Which value type in C# allows you to know the references to all of the variables?
 - a. Reference type.
 - b. Object type.
 - c. Pointer type.
 - d. Value types.
5. Which value type in C# does not allow you to store data directly?
 - a. Object type.
 - b. Reference type.
 - c. Pointer type.
 - d. Value type.



Extension Activity

It's time to learn something new.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about kinds of data in C#
- Take notes about the information you read.
- Make a poster showing the different kinds of data in C#.
- Explain how some of these data work in C#.
- Use simple present tense of the verb allow to talk about possibility.
- Share your poster with a partner.

Self-Assessment

Yes

No

- I search information about different kinds of data in C#
- I produce a poster to show some kinds of data in C#
- I explain how some data work in C#

☐ ☐
☐ ☐
☐ ☐


Target Words

- **get your head around something** (expression): to understand
e.g. I still can't get my head around this C# stuff!
- **floating point number**(noun): a real number with no fixed place for the decimal point
e.g. Let's start with value types; they allow us to store data directly, such as numbers, alphabets, and floating point numbers, and they can be directly assigned a value.
- **reference type** (noun): it refers to the memory location
e.g. The reference type does not store data directly but contains references to all of the variables.
- **indexed file** (noun): a part of the operating system that deals with how data is stored
e.g. Oh so it's like an indexed file system that allows you to look up the data you need.
- **convert** (verb): to change data into another format
e.g. However, before assigning values, it needs converting.
- **dynamic type** (noun): you can store any type value. Type checking for this type takes place during run-time.
e.g. The dynamic type is also useful as you can store any data type inside it.



Learning Points

Simple Present Tense of the Verb **allow** to Talk About Permitting Possibility

We can use the verb **allow** to talk about a thing, person, or event that permits something or someone to perform an activity.

We can use the verb **allow** when we are talking about permitting or giving the possibility to someone or something to execute or perform an action.

Form: subject + **allow** + direct object + infinitive form of the verb

Note that if the subject corresponds to the third person singular, you have to add **-s** to the verb.

Examples:

*This command **allows** you to represent a Boolean value.*

In the above mentioned case, there is something (the command) that permits the person (who is implicit) to represent a specific value in his programming code.

*She **allows** her employees to work with C++ and C#.*

In this new case, it's a person who permits other people to do something. This time, we can see that the sentence talks more about giving permission than helping or giving people the necessary tools to perform the mentioned action.

Examples

- *The index system allows you to look up for data you need*
- *Value type allows us to store data directly*





Learning Points

Value types in C#.

In the table below, you will find a list of the most common value types in C#.

Type	Represents	Range	Default Value
bool	Boolean value	True or False	False
byte	8-bit unsigned integer	0 to 255	0
char	16-bit Unicode character	U +0000 to U +ffff	'\0'
decimal	128-bit precise decimal values with 28-29 significant digits	$(-7.9 \times 10^{28}$ to $7.9 \times 10^{28}) / 100$ to 28	0.0M
double	64-bit double-precision floating point type	$(+/-)5.0 \times 10^{-324}$ to $(+/-)1.7 \times 10^{308}$	0.0D
float	32-bit single-precision floating point type	-3.4×10^{38} to $+ 3.4 \times 10^{38}$	0.0F
int	32-bit signed integer type	-2,147,483,648 to 2,147,483,647	0
long	64-bit signed integer type	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
sbyte	8-bit signed integer type	-128 to 127	0
short	16-bit signed integer type	-32,768 to 32,767	0
uint	32-bit unsigned integer type	0 to 4,294,967,295	0
ulong	64-bit unsigned integer type	0 to 18,446,744,073,709,551,615	0
ushort	16-bit unsigned integer type	0 to 65,535	0